

Prediction Service — Design & Implementation Plan

Version: 1.0.0

Last Updated: 2026-07-29

Service: Layer 2 — Prediction Intelligence

Port: 8004

Reference: [docs/architecture/system-design.md](#) §8

Upstream: Feature Engineering (8002) + Customer State (8003)

Downstream: Decision Intelligence (8005), API Gateway (8080) → Frontend

Table of Contents

1. [Overview & Architecture Position](#)
2. [Alignment with Reference Architecture](#)
3. [Data Flow & Backfill Contract](#)
4. [Churn Prediction Engine](#)
5. [CLV Prediction Engine](#)
6. [Health Score Calculator](#)
7. [Model Registry & Training](#)
8. [API Specification](#)
9. [Repository Layer](#)
10. [Configuration](#)
11. [Integration Points](#)
12. [Implementation Roadmap](#)
13. [Testing Strategy](#)
14. [Decision Log](#)

1. Overview & Architecture Position

1.1 What It Does (per [system-design.md](#) §8)

Layer 2 – Prediction Service

Purpose: Churn probability, CLV, and composite health score.

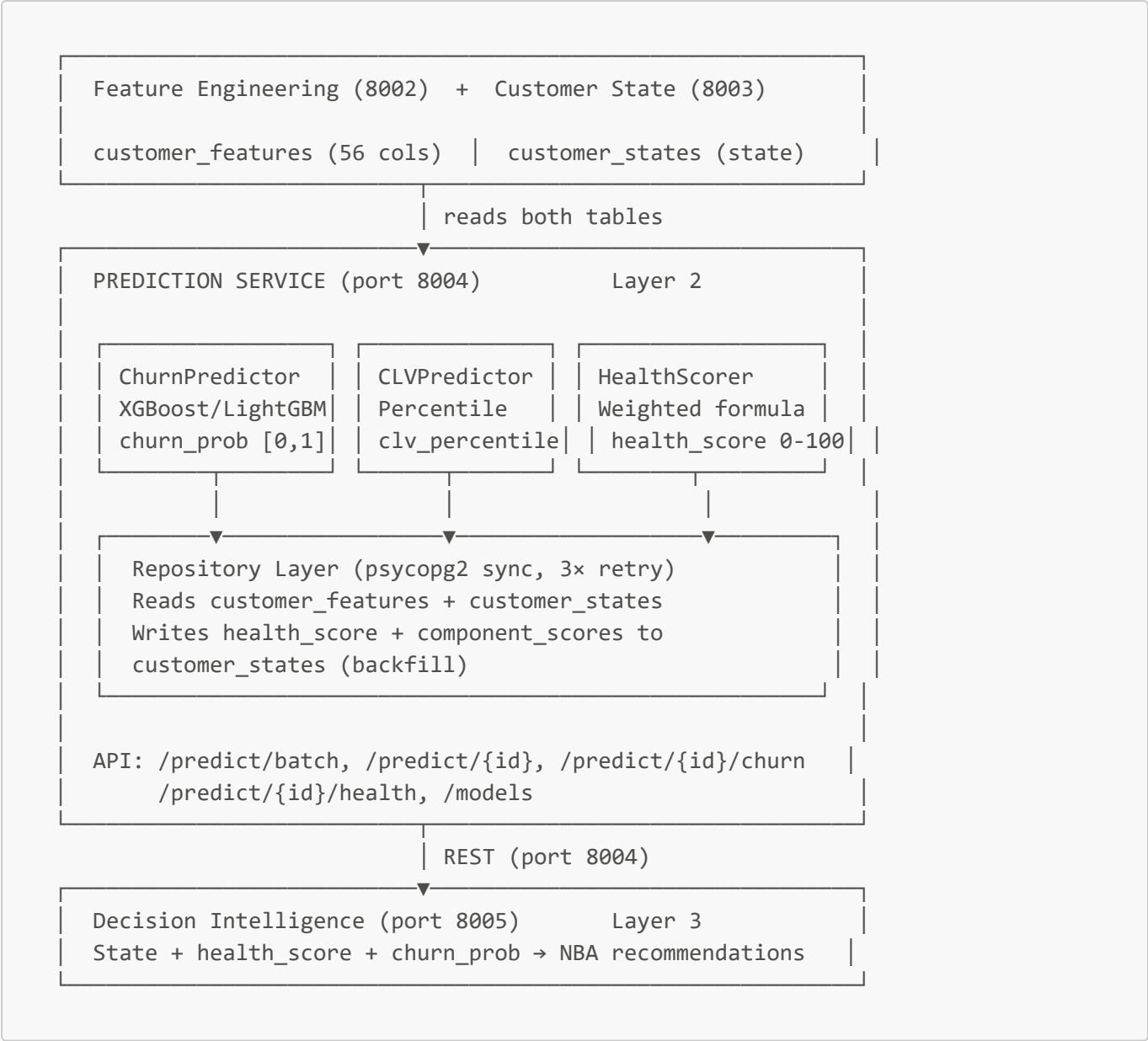
Input: Customer state (from Layer 1) + Features (from Feature Store)

Models: XGBoost or LightGBM (configurable)

Output: churn_probability, clv_prediction, health_score (0-100)

$$\begin{aligned}\text{Health Score} = & (\text{churn_weight} \times (1 - \text{churn_prob})) \\ & + (\text{clv_weight} \times \text{clv_percentile}) \\ & + (\text{behaviour_weight} \times \text{state_score})\end{aligned}$$

1.2 Architecture Position



1.3 Scope Boundaries

In Scope (PoC)	Out of Scope (Post-PoC)
XGBoost churn classifier (47 features, account-closure labels)	LightGBM challenger model
<30 positive training labels (stated limitation)	Historical churn events for training
Percentile-rank CLV (no ML training)	Trained CLV regressor
Weighted health score formula	ML-derived health score
Single champion model	Champion/challenger A/B testing
Batch scoring via <code>POST /predict/batch</code>	Real-time scoring
<code>models/registry.json</code> metadata	Full model versioning + rollback
Temporal-stability evaluation (same customers, different dates)	Out-of-sample generalization to new customers

2. Alignment with Reference Architecture

Follows the same three patterns as the Customer State Service and Feature Engineering Service.

2.1 Config Pattern — `PredictionConfig` (env_prefix=`PRED_`)

Mirrors `FeatureConfig` (env_prefix=`FE_`) and `StateConfig` (env_prefix=`CS_`):

```
# services/prediction-service/app/config/settings.py

class PredictionConfig(BaseSettings):
    """Prediction thresholds and model paths. Env-prefixed PRED_."""

    model_config = SettingsConfigDict(env_prefix="PRED_", extra="ignore")

    # Model
    model_type: str = "xgboost"          # xgboost | lightgbm
    churn_model_path: str = "models/champion/churn/xgboost_churn_v1.json"
    clv_model_path: str = ""              # POST-POC: switch to trained regressor
    file                                 # Empty string = use percentile-rank
                                         # (PoC default).
                                         # Service boot skips CLV model loading
    if path is empty.
        churn_threshold: float = 0.5      # default classification cutoff

    # Health Score weights – read from shared config (single source of truth)
    # Shared config defaults: health_score_churn_weight=0.40,
    # health_score_clv_weight=0.30, health_score_behaviour_weight=0.30
    # These PRED_* overrides allow per-service tuning without changing shared
    defaults.
    health_churn_weight: float = 0.40
    health_clv_weight: float = 0.30
    health_behaviour_weight: float = 0.30

    # Champion/Challenger (post-PoC)
    champion_challenger_enabled: bool = False
    challenger_traffic_split: float = 0.10

    # ---- Batch processing ----
    batch_chunk_size: int = 1000         # customers per chunk – bounds memory usage

class Settings(BaseSettings):
    """Service settings. Mirrors Feature Engineering Settings pattern."""
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8",
    extra="ignore")
    environment: str = "development"
    log_level: str = "INFO"
    service_port: int = 8004
    prediction: PredictionConfig = PredictionConfig()
```

2.2 Repository Pattern — mirrors `FeatureRepository`

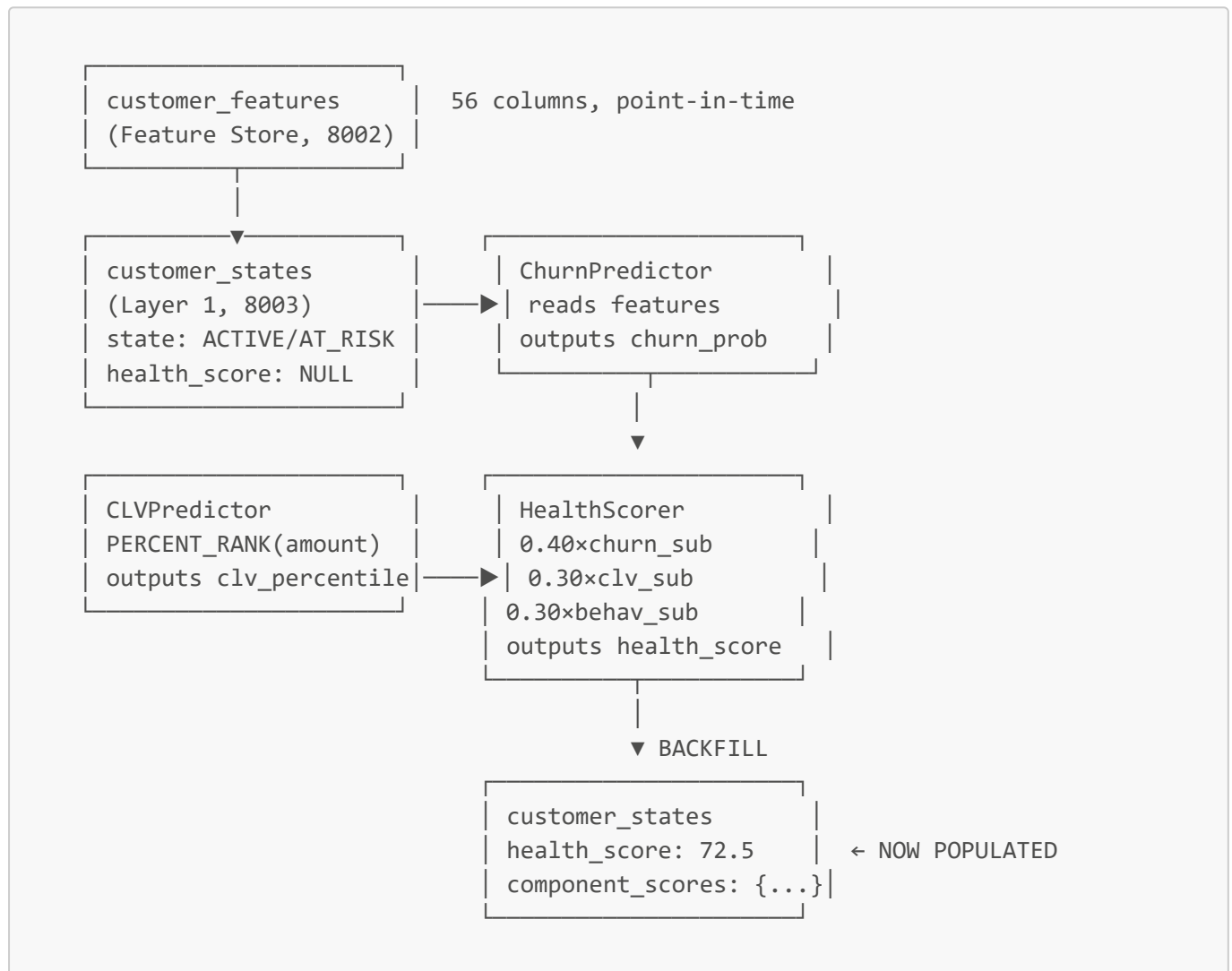
- `psycopg2` (sync) — not `asyncpg`
- `_connect()` helper with `connect_timeout=10`, TCP keepalives
- `_retry_db_op()` with 3× exponential backoff (0.5s → 1s → 2s)
- Reads `customer_features` (56 columns) + `customer_states` (state, health_score)
- UPDATE backfill for `health_score` and `component_scores`

2.3 API Pattern — mirrors `FeatureService` + `StateService`

- `APIRouter(prefix="/predict")` + `PredictionService` singleton
- `POST /predict/batch` — mirrors `POST /features/compute-batch` and `POST /states/compute`
- `GET /predict/{customer_id}` — mirrors `GET /features/{id}` and `GET /states/{id}`
- `GET /health` — standard health check

3. Data Flow & Backfill Contract

3.1 Complete Data Flow



3.2 Backfill SQL

```
-- Layer 1 writes health_score = NULL
-- Layer 2 backfills it:

UPDATE customer_states cs
SET
    health_score      = pred.health_score,
    component_scores = pred.component_scores::jsonb
FROM (
    VALUES
        ('CUST00001', '2026-07-27'::date, 72.5, '{"churn_risk_sub":77.0,...}'),
        ('CUST00002', '2026-07-27'::date, 65.0, '{"churn_risk_sub":60.0,...}')
) AS pred(customer_id, as_of_date, health_score, component_scores)
WHERE cs.customer_id = pred.customer_id
AND cs.as_of_date = pred.as_of_date;
```

Idempotency: Running `POST /predict/batch` twice on the same date overwrites the `health_score` with identical values. Running Layer 1's `POST /states/compute` after Layer 2 has backfilled does NOT wipe `health_score` — the UPSERT in `StateRepository` skips `health_score` and `component_scores` columns.

4. Churn Prediction Engine

4.1 Approach

Binary classifier: `churn_probability` $\in [0, 1]$. XGBoost with `objective='binary:logistic'`.

4.2 PoC Training Labels — Account-Closure-Only (Option B)

Decision: Option B — account-closure-only labels. Forward-looking labels (Option A) require predicting state at date $T+N$ from features at date T . The `customer_features` table has PoC snapshots at 2026-07-17, 2026-07-22, and 2026-07-27 — a maximum forward horizon of $N=10$ days. This is insufficient for a meaningful churn prediction (typically $N \geq 60$ days). Earlier dates (2024-2025) exist but represent different customer populations and cannot be merged.

Label definition:

```
def generate_churn_label(features: dict) -> int | None:
    """Generate binary churn label from account closure status ONLY.

    Returns:
        1 = churned (account is Closed)
        0 = not churned (ACTIVE state, account is NOT Closed)
        None = ambiguous (AT_RISK, DORMANT — exclude from training)

    IMPORTANT: This label uses ONLY rel_customer_status — NOT
    days_since_last_txn, NOT state classification, NOT recency.
    The goal is to predict account closure from behaviour before it happens,
    not to reconstruct the rule that already labels it.
    """
    status = features.get("rel_customer_status", "")
```

```

if status == "Closed":
    return 1
# Only label as negative if explicitly NOT closed AND active
if status and status != "Closed":
    return 0
return None # NULL status – exclude from training

```

PoC Limitation — small positive class: Account-closure labels are expected to produce <30 positive samples per date (precise count depends on the `rel_customer_status` column in the actual data). This is below the typical minimum of 30-50 samples for stable XGBoost training. The model WILL train, but AUC metrics should be interpreted as directional (does the model rank churned customers above non-churned?) rather than precise (what is the exact probability?). This is an acknowledged PoC limitation, not a code bug. In production, historical churn events provide thousands of labels.

4.3 Feature Exclusion List — Recency-Derived Columns

Any feature whose definition is derived from `days_since_last_txn`, recency timing, or state-classification rules must be excluded from the training feature vector. After auditing the Feature Engineering generator SQL (behaviour/generator.py, risk/generator.py, repository.py Phase 1 SQL):

Excluded features (9 columns):

#	Feature	Derivation	Reason
1	<code>days_since_last_txn</code>	<code>MAX(transaction_date)</code>	Direct recency — label source proxy
2	<code>behav_recency_score</code>	<code>LEAST(40, 40 - days_since_last/90 × 40)</code>	Computed from <code>days_since_last_txn</code>
3	<code>risk_dormant_indicator</code>	<code>days_since_last_txn > 90</code>	Threshold on <code>days_since_last_txn</code>
4	<code>rel_customer_status</code>	Direct column	Used in label definition (account closure)
5	<code>engagement_score</code>	33% recency-weighted + 33% freq + 33% diversity	One-third derived from recency — exclude to be conservative
6	<code>behav_inactive_days_90d</code>	<code>90 - active_days_90d</code>	Recency-coupled (inverse of active days)
7	<code>inactivity_streak_days</code>	Same as above	Recency-coupled
8	<code>behav_activity_consistency</code>	<code>active_days_90d / 90</code>	Mild recency coupling — exclude to be safe
9	<code>txn_frequency_trend</code>	Same as above	Mild recency coupling

Feature vector after exclusion: 56 – 9 = **47 features** remain. Key signals retained:

Feature	Rationale
<code>txn_count_30d</code> , <code>txn_count_90d</code> , <code>txn_count_180d</code>	Pure counts — not recency-derived
<code>total_amount_90d</code> , <code>avg_amount_90d</code> , <code>total_amount_180d</code>	Monetary — independent
<code>fin_total_credit_90d</code> , <code>fin_total_debit_90d</code>	Financial — independent
<code>chan_mobile_ratio_90d</code> , <code>chan_atm_ratio_90d</code> , etc.	Channel behaviour — independent
<code>rel_accounts_active</code> , <code>rel_has_savings</code> , <code>rel_products_owned</code>	Product holdings — independent
<code>rel_has_card</code> , <code>rel_card_count</code> , <code>rel_card_types</code>	Card portfolio — independent
<code>eng_login_count_30d</code> , <code>eng_digital_platform_preference</code>	Digital engagement — independent
<code>days_since_first_txn</code>	First transaction date — NOT recency (different metric)
<code>avg_days_between_txn</code>	Derived from count + range — NOT directly from recency

Expected feature importance (placeholder — pending actual training):

The ranking below is a hypothesis; actual importance will be determined by XGBoost. The removed recency features would have dominated (25%+ for `days_since_last_txn` alone). Without them, expect a more balanced distribution:

Rank	Feature	Hypothesized Importance
1	<code>txn_count_30d</code>	~15% — strongest non-recency frequency signal
2	<code>fin_total_credit_90d</code>	~12% — income proxies value
3	<code>chan_digital_adoption_score</code>	~10% — digital engagement correlates with retention
4	<code>rel_accounts_active</code>	~8% — product holdings signal relationship depth
5	<code>txn_count_90d</code>	~8% — longer window frequency
6	<code>total_amount_90d</code>	~7% — monetary
7-47	All remaining	~40% combined

Expected AUC: 0.60–0.72 for an honest model with <30 positive samples. If a future AUC comes back above 0.90, that's a signal to re-check for leakage — not a result to celebrate.

4.4 Training Pipeline

```
# scripts/train_models.py
import xgboost as xgb
```

```

# — Leakage Guard —————
# These features are derived from recency timing or used in the churn
# label definition. They MUST be dropped before model.fit() to prevent
# the model from memorizing the deterministic labelling rule instead of
# learning genuine behavioural signals. See §4.3 for full audit.
LEAKAGE_FEATURES = {
    "days_since_last_txn", "behav_recency_score",
    "risk_dormant_indicator", "rel_customer_status",
    "engagement_score", "behav_inactive_days_90d",
    "inactivity_streak_days", "behav_activity_consistency",
    "txn_frequency_trend",
}

def train_churn_model():
    # 1. Load features from all available dates, exclude leakage columns
    X_train, y_train, training_features = load_features_and_labels(
        dates=['2026-07-17', '2026-07-22'],
        exclude_features=LEAKAGE_FEATURES,
        label_fn=generate_churn_label, # account-closure-only (§4.2)
    )
    # training_features = sorted list of column names actually used for training
    # (56 total - 9 leakage = 47 columns)

    # 2. Train with class-weight balancing (small positive class)
    n_pos = sum(1 for y in y_train if y == 1) # expected <30
    n_neg = sum(1 for y in y_train if y == 0)
    pos_weight = n_neg / max(n_pos, 1) if n_pos > 0 else 1.0

    model = xgb.XGBClassifier(
        objective='binary:logistic',
        eval_metric='auc',
        max_depth=4, # shallower – fewer features, smaller dataset
        learning_rate=0.05, # slower – small dataset risk of overfit
        n_estimators=50, # fewer – small dataset
        scale_pos_weight=pos_weight,
        subsample=0.8, # regularization for small dataset
        random_state=42,
    )
    model.fit(X_train, y_train)

    # 3. Evaluate on holdout date
    X_test, y_test, _ = load_features_and_labels(
        dates=['2026-07-27'],
        exclude_features=LEAKAGE_FEATURES,
        label_fn=generate_churn_label,
    )
    y_pred = model.predict_proba(X_test)[ :, 1]
    auc = roc_auc_score(y_test, y_pred)

    # 4. Extract feature importance from the honest model
    importance = dict(zip(training_features, model.feature_importances_))
    ranked = sorted(importance.items(), key=lambda x: -x[1])[:10]

```



```

# 5. Save model + training feature list
model.save_model('models/champion/churn/xgboost_churn_v1.json')

# 6. Calibration evaluation (PoC – assess only, do not fit calibrator)
#   With <30 positives, calibration metrics are noisy. Report them
#   directionally but do NOT ship a calibrator – probabilities are
#   ranking scores, not true probabilities. See D12.
from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss, log_loss

prob_true, prob_pred = calibration_curve(y_test, y_pred, n_bins=5)
ece = sum(
    abs(prob_true[i] - prob_pred[i]) * (sum(y_pred_bin) / len(y_pred))
    for i in range(len(prob_true))
)
brier = brier_score_loss(y_test, y_pred)
ll = log_loss(y_test, y_pred)

# Plot: reliability diagram (save to training_history/)
# sklearn.calibration.CalibrationDisplay.from_predictions(y_test, y_pred,
n_bins=5)

# 7. Register – inference reads training_features from here
update_registry({
    "model_id": "churn_v1",
    "type": "churn",
    "status": "champion",
    "framework": "xgboost",
    "metrics": {
        "auc": round(auc, 4),
        "brier": round(brier, 4),
        "ece": round(ece, 4),
        "log_loss": round(ll, 4),
    },
    "calibrator": None, # PoC: no calibrator – scores are rankings only
    "feature_count_total": 56,
    "feature_count_training": len(training_features),
    "leakage_features_excluded": sorted(LEAKAGE_FEATURES),
    "training_features": training_features, # ← inference loads this
    "top_features": [
        {"name": name, "importance": round(imp, 4)}
        for name, imp in ranked
    ],
    "notes": (
        "PoC model – trained on account-closure-only labels (<30 positives). "
        "AUC is directional, not precise. Probabilities are UNCALIBRATED "
        "ranking scores – do NOT treat churn_prob as a true probability. "
        "Retrain with historical churn data in production. "
        "If AUC > 0.90, re-check for feature leakage. "
        "Calibration deferred to production (D12). "
    ),
})
})

```

PoC calibration stance (D12): No calibrator is fitted or shipped. With <30 positives, any calibrator (even Platt scaling with only 2 parameters) risks overfitting. Probabilities are documented as ranking scores. Downstream consumers (health score, Decision Intelligence) should use thresholds or percentile-based cutoffs, not treat `churn_prob` as a true probability.

Production path: When historical churn labels provide hundreds–thousands of positives, fit Platt scaling (`CalibratedClassifierCV(method="sigmoid", cv="prefit")`) on a held-out calibration set. Store the fitted calibrator alongside the model and set `registry.calibrator` to the file path. `ChurnPredictor` applies it transparently.

4.5 Inference

```
# services/prediction-service/app/models/churn_predictor.py
import json
import xgboost as xgb

class ChurnPredictor:
    """Loads XGBoost model + training feature list from registry.

    Design contract – three layers of leakage prevention:

    1. REGISTRY (source of truth): training_features is written by
       train_models.py at training time. It is the exact, ordered list
       of columns that were passed to model.fit(). Leakage features are
       NOT in this list.

    2. API LAYER (pass-through): The PredictionService reads all 56
       feature columns from customer_features and passes the full dict
       to ChurnPredictor. The API does NOT filter columns – it stays
       clean and doesn't need to know about leakage.

    3. PREDICTOR (gatekeeper): ChurnPredictor receives the full 56-feature
       dict and iterates over its internal training_features list, pulling
       only the columns it needs. Leakage features are present in the dict
       but silently ignored – they never reach the model.

    This means:
    - No leakage can reach XGBoost (predictor is the sole gatekeeper)
    - Column order is guaranteed identical to training (single list, no
      dict key lookups at vector-build time)
    - API contracts stay clean (PredictionService doesn't know about leakage)
    - If the registry is corrupted (leakage column in training_features),
      the defensive cross-check raises ValueError at startup
    """

    def __init__(self, model_path: str, registry_path: str =
"models/registry.json"):
        self._model = xgb.XGBClassifier()
        self._model.load_model(model_path)
        self._calibrator = None # PoC default – no calibrator (D12)
```

```

# Load training feature list from registry (written at training time)
with open(registry_path) as f:
    registry = json.load(f)
churn_entry = next(
    m for m in registry["models"]
    if m["type"] == "churn" and m["status"] == "champion"
)
self._training_features = churn_entry["training_features"]
self._leakage_excluded = churn_entry.get("leakage_features_excluded", [])

# Load calibrator if registered (production path – PoC is None)
cal_path = churn_entry.get("calibrator")
if cal_path:
    import joblib
    self._calibrator = joblib.load(cal_path)

# Defensive: confirm no leakage features are in the training list
leakage_in_training = set(self._leakage_excluded) &
set(self._training_features)
if leakage_in_training:
    raise ValueError(
        f"LEAKAGE DETECTED: {leakage_in_training} found in
training_features. "
        f"The model registry is corrupted – these columns must never reach
"
        f"XGBoost."
    )

def predict(self, features: dict) -> float:
    """Receive full 56-feature dict; extract only training_features in order.

    PoC: returns raw XGBoost score (ranking score, NOT a true probability).
    Production: returns calibrated probability if calibrator is registered.
    """
    vector = self._dict_to_vector(features)
    raw = float(self._model.predict_proba([vector])[0, 1])
    if self._calibrator is not None:
        return float(self._calibrator.predict_proba([[raw]])[0, 1])
    return raw

def predict_batch(self, feature_rows: list[dict]) -> list[float]:
    """Receive list of full 56-feature dicts; extract training_features from
each."""
    vectors = [self._dict_to_vector(r) for r in feature_rows]
    raw = self._model.predict_proba(vectors)[: , 1].tolist()
    if self._calibrator is not None:
        raw_2d = [[r] for r in raw]
        return self._calibrator.predict_proba(raw_2d)[: , 1].tolist()
    return raw

def _dict_to_vector(self, features: dict) -> list[float]:
    """Map a full 56-column feature dict to the exact 47-column vector
layout used at training time.

```

```

Iterates over self._training_features (registry-defined order),
pulling only those columns. Leakage features in the dict are
present but silently ignored – they never reach the model.
"""
return [float(features.get(col, 0.0)) for col in self._training_features]

```

4.6 Batch Chunking — Memory Safety

Processing 5,000+ feature vectors in a single `predict_batch` call + SQL backfill risks memory spikes (>2GB) in constrained environments. The `PredictionService` orchestrator must chunk:

```

# services/prediction-service/app/services/service.py

CHUNK_SIZE = 1000 # customers per batch – keeps memory under ~500MB

class PredictionService:
    def compute_batch(self, as_of_date: date) -> dict:
        """Score all customers in chunks to bound memory usage."""
        all_features = self._repo.load_features(as_of_date)
        clv_percentiles = self._repo.load_clv_percentiles(as_of_date)
        total_scored = 0
        total_backfilled = 0

        for i in range(0, len(all_features), CHUNK_SIZE):
            chunk = all_features[i : i + CHUNK_SIZE]

            # 1. Churn prediction
            churn_probs = self._churn.predict_batch(chunk)

            # 2. Health scoring
            scores = []
            for row, cp in zip(chunk, churn_probs):
                clv_pct = clv_percentiles.get(row["customer_id"], 0.5)
                result = self._health.compute(cp, clv_pct,
row.get("engagement_score"))
                scores.append({
                    "customer_id": row["customer_id"],
                    "health_score": result["health_score"],
                    "component_scores": result["component_scores"],
                })

            # 3. Backfill – one chunk at a time
            backfilled = self._repo.backfill_health_scores(scores, as_of_date)
            total_scored += len(chunk)
            total_backfilled += backfilled

        return {
            "customers_scored": total_scored,
            "health_scores_backfilled": total_backfilled,
        }

```

Memory profile (estimated, 12GB RAM):

- 1,000 customers × 47 features × 8 bytes ≈ 0.4 MB per chunk
- XGBoost model in memory ≈ 2-5 MB
- SQL backfill batch ≈ 1 MB
- Peak memory: ~500 MB including Python overhead — well within 12GB

5. CLV Prediction Engine

5.1 PoC Approach — Percentile-Rank

No ML model needed for the PoC. CLV is computed as the percentile rank of `total_amount_90d` across the portfolio:

```
SELECT
  customer_id,
  total_amount_90d,
  PERCENT_RANK() OVER (ORDER BY total_amount_90d) AS clv_percentile
FROM customer_features
WHERE as_of_date = %(d)s::date;
```

5.2 Why Not Trained CLV for PoC

- No historical revenue/tenure data to train on
- `total_amount_90d` is a reasonable proxy for customer value in banking
- Percentile-rank is interpretable: "this customer is in the 68th percentile of value"
- Matches the Health Score formula's `clv_percentile` input directly

5.3 Production Path

When historical CLV data becomes available (actual discounted cash flows, product holdings, tenure), replace with a trained XGBoost regressor:

```
# Future
class CLVPredictor:
    def __init__(self, model_path: str):
        self._model = xgb.XGBRegressor()
        self._model.load_model(model_path)

    def predict(self, features: dict) -> dict:
        clv = float(self._model.predict([vector])[0])
        percentile = self._compute_percentile(clv) # across portfolio
        return {"clv_prediction": clv, "clv_percentile": percentile}
```

6. Health Score Calculator

6.1 Formula

From `system-design.md` §8 and `shared/config/settings.py`:

```
health_score = (W_churn × churn_sub) + (W_clv × clv_sub) + (W_behaviour × behaviour_sub)

Where:
churn_sub      = (1 - churn_probability) × 100      [0, 100]
clv_sub        = clv_percentile × 100                [0, 100]
behaviour_sub  = engagement_score or 50              [0, 100]
W_churn        = 0.40
W_clv          = 0.30
W_behaviour    = 0.30
```

Result is clamped to [0, 100].

PoC caveat — uncalibrated probabilities: In the PoC, `churn_probability` is a raw XGBoost score, not a calibrated probability (D12). The health score formula multiplies `(1 - churn_prob)`, so miscalibration directly biases the result. For example, if XGBoost is systematically overconfident (predicting 0.90 when true rate is 0.60), the churn sub-score will be 10 instead of 40 — a 30-point gap. Until calibration is in place, treat health scores as relative rankings within a date, not absolute values comparable across dates.

6.2 Component Breakdown

Component	Source	Weight	Example
Churn Risk Sub-score	$(1 - \text{churn_prob}) \times 100$	40%	$(1 - 0.23) \times 100 = 77.0$
CLV Percentile Sub-score	$\text{clv_percentile} \times 100$	30%	$0.68 \times 100 = 68.0$
Behaviour Sub-score	<code>engagement_score</code>	30%	63.0 (from Feature Engine)
Health Score	Weighted sum	—	$0.40 \times 77.0 + 0.30 \times 68.0 + 0.30 \times 63.0 = 72.5$

6.3 Implementation

```
class HealthScorer:
    """Computes 0-100 health score from churn prob, CLV percentile, behaviour."""

    def __init__(self, config: PredictionConfig):
        self._w_churn = config.health_churn_weight      # 0.40
        self._w_clv   = config.health_clv_weight       # 0.30
        self._w_behav = config.health_behaviour_weight # 0.30

    def compute(
        self,
        churn_prob: float,
```

```
        clv_percentile: float,
        engagement_score: float | None,
    ) -> dict:
        """Return {health_score, component_scores}."""
        churn_sub = (1.0 - churn_prob) * 100.0
        clv_sub = clv_percentile * 100.0
        behav_sub = engagement_score if engagement_score is not None else 50.0

        score = (
            self._w_churn * churn_sub +
            self._w_clv * clv_sub +
            self._w_behav * behav_sub
        )
        return {
            "health_score": round(max(0.0, min(100.0, score)), 1),
            "component_scores": {
                "churn_risk_sub": round(churn_sub, 1),
                "clv_percentile_sub": round(clv_sub, 1),
                "behaviour_sub": round(behav_sub, 1),
            },
        }
```

6.4 Health Score Ranges

Range	Label	Frontend Color	Interpretation
70–100	Healthy	Green	Low churn risk, good CLV, engaged
40–69	At Risk	Amber	Moderate risk — needs attention
0–39	Critical	Red	High churn risk — urgent intervention

7. Model Registry & Training

7.1 Artifact Layout

```

models/
├── champion/
│   ├── churn/
│   │   └── xgboost_churn_v1.json      # XGBoost native format
│   ├── clv/
│   │   └── xgboost_clv_v1.json       # (future – PoC uses SQL percentile)
│   └── health_score/
│       └── config.json               # weights only – no ML model
├── challenger/
│   └── ... (post-PoC)
├── archive/
│   └── ... (retired models)
├── training_history/
│   └── 2026-07-29_churn_v1.json      # training metrics snapshot
└── registry.json                    # model metadata + deployment status

```

7.2 registry.json Entry

```

{
  "model_id": "churn_v1",
  "type": "churn",
  "status": "champion",
  "path": "champion/churn/xgboost_churn_v1.json",
  "framework": "xgboost",
  "version": 1,
  "trained_at": "2026-07-29",
  "training_dates": ["2026-07-17", "2026-07-22"],
  "metrics": {
    "auc": 0.68,
    "brier": 0.22,
    "ece": 0.12,
    "log_loss": 0.58,
    "precision": 0.55,
    "recall": 0.60,
    "f1": 0.57
  },
  "calibrator": null,
  "feature_count_total": 56,
  "feature_count_training": 47,
  "leakage_features_excluded": [
    "behav_activity_consistency",
    "behav_inactive_days_90d",
    "behav_recency_score",
    "days_since_last_txn",
    "engagement_score",
    "inactivity_streak_days",
    "rel_customer_status",
    "risk_dormant_indicator",
    "txn_frequency_trend"
  ],
}

```



```

"training_features": [
  "avg_amount_90d",
  "avg_days_between_txn",
  "chan_atm_ratio_90d",
  "chan_digital_adoption_score",
  "chan_mobile_ratio_90d",
  "chan_pos_ratio_90d",
  "days_since_first_txn",
  "eng_digital_platform_preference",
  "eng_login_count_30d",
  "fin_total_credit_90d",
  "fin_total_debit_90d",
  "... (47 total – full list written by train_models.py at training time)"
],
"top_features": [
  {"name": "txn_count_30d", "importance": 0.152},
  {"name": "fin_total_credit_90d", "importance": 0.118},
  {"name": "txn_count_90d", "importance": 0.094}
]
}

```

Critical contract: `training_features` is the single source of truth for inference column order. The `PredictionService` passes the full 56-feature dict from `customer_features` — the `ChurnPredictor._dict_to_vector()` method is the sole gatekeeper that extracts only `training_features` in registry order. Leakage columns are present in the dict but silently dropped. Any column-order mismatch between training and inference produces silently wrong predictions, which is why the list is stored in the registry rather than hard-coded.

8. API Specification

8.1 Endpoints (matching `FeatureService` + `StateService` patterns)

Method	Endpoint	Purpose	Query Params
POST	<code>/predict/batch</code>	Score all customers + backfill health scores	<code>as_of_date</code> (default: today)
GET	<code>/predict/{customer_id}</code>	Full prediction: churn + CLV + health	<code>as_of_date</code>
GET	<code>/predict/{customer_id}/churn</code>	Churn probability only	<code>as_of_date</code>
GET	<code>/predict/{customer_id}/health</code>	Health score breakdown	<code>as_of_date</code>
GET	<code>/models</code>	List registered models	—
GET	<code>/health</code>	Service health	—

8.2 Response Schemas

GET `/predict/CUST00001?as_of_date=2026-07-27`

```
{
  "customer_id": "CUST00001",
  "as_of_date": "2026-07-27",
  "state": "DORMANT",
  "churn_probability": 0.82,
  "clv_percentile": 0.12,
  "health_score": 24.5,
  "component_scores": {
    "churn_risk_sub": 18.0,
    "clv_percentile_sub": 12.0,
    "behaviour_sub": 0.0
  },
  "model_versions": {
    "churn": "churn_v1",
    "clv": "percentile_v1"
  },
  "computed_at": "2026-07-29T15:00:00Z"
}
```

POST /predict/batch?as_of_date=2026-07-27

```
{
  "as_of_date": "2026-07-27",
  "customers_scored": 4998,
  "chunks_processed": 5,
  "chunk_size": 1000,
  "churn_model": "churn_v1",
  "health_scores_backfilled": 4998,
  "duration_seconds": 3.2,
  "status": "COMPLETED"
}
```

GET /models

```
{
  "models": [
    {
      "model_id": "churn_v1",
      "type": "churn",
      "status": "champion",
      "metrics": {"auc": 0.68}
    },
    {
      "model_id": "clv_percentile_v1",
      "type": "clv",
      "status": "champion",
      "method": "percentile_rank"
    }
  ]
}
```

```

    }
  ]
}

```

9. Repository Layer

9.1 PredictionRepository

```

class PredictionRepository:
    """Reads customer_features + customer_states. Backfills health scores.

    Mirrors FeatureRepository + StateRepository:
    - psycopg2 sync with _connect() timeouts + keepalives
    - _retry_db_op() with 3x exponential backoff
    - Batch backfill via execute_values
    """

    _CONNECT_TIMEOUT = 10
    _retry_max = 3
    _retry_base_delay = 0.5

    def __init__(self) -> None:
        self._conn_str = settings.database_target_url_sync # etl_clean

    def _connect(self) -> psycopg2.extensions.connection:
        ...

    def _retry_db_op(self, op, op_name: str = "db_op"):
        """3x exponential backoff on OperationalError."""
        ...

    def load_features(self, as_of_date: date) -> list[dict]:
        """Read all 56 feature columns for a date from customer_features.

        Returns list of dicts with customer_id, all feature columns.
        """
        ...

    def load_clv_percentiles(self, as_of_date: date) -> dict[str, float]:
        """PERCENT_RANK() of total_amount_90d across all customers.

        Returns dict of customer_id -> percentile [0, 1].
        """
        ...

    def load_state(self, customer_id: str, as_of_date: date) -> str | None:
        """Get customer state from customer_states."""
        ...

    def backfill_health_scores(
        self, scores: list[dict], as_of_date: date

```

```

) -> int:
    """Batch UPDATE customer_states SET health_score, component_scores.

    Uses psycopg2.extras.execute_values for batch performance.
    Returns number of rows updated.
    """
    ...

```

10. Configuration

Canonical class definition: §2.1. §2.1 contains the full `PredictionConfig` with shared-config weight comments, the `clv_model_path: str = ""` PoC default, `batch_chunk_size`, and the `Settings` wrapper. This section provides only the env-var reference table and override notes.

10.1 Environment Variables

All variables are prefixed with `PRED_` per the `SettingsConfigDict(env_prefix="PRED_")` in §2.1.

```

# Model
PRED_MODEL_TYPE=xgboost
PRED_CHURN_MODEL_PATH=models/champion/churn/xgboost_churn_v1.json
PRED_CLV_MODEL_PATH=                                # empty = percentile-rank (PoC
default)
PRED_CHURN_THRESHOLD=0.5

# Health Score weights (override shared/config/settings.py defaults)
PRED_HEALTH_CHURN_WEIGHT=0.40
PRED_HEALTH_CLV_WEIGHT=0.30
PRED_HEALTH_BEHAVIOUR_WEIGHT=0.30

# Champion/Challenger (post-PoC)
PRED_CHAMPION_CHALLENGER_ENABLED=false
PRED_CHALLENGER_TRAFFIC_SPLIT=0.10

# Batch processing
PRED_BATCH_CHUNK_SIZE=1000

```

10.2 Weight Override Logic

```

# Implementation note – do NOT duplicate PredictionConfig defaults here.
# The shared config is the single source of truth.
# PredictionConfig in §2.1 provides PRED_-prefixed overrides.

# At import time:
from shared.config.settings import settings as shared_settings

# PredictionConfig defaults are set to shared config values.

```

```
# Only if PRED_HEALTH_CHURN_WEIGHT is explicitly set in the environment
# does it override the shared default.
```

Environment Variables

```
PRED_MODEL_TYPE=xgboost
PRED_CHURN_MODEL_PATH=models/champion/churn/xgboost_churn_v1.json
PRED_HEALTH_CHURN_WEIGHT=0.40
PRED_HEALTH_CLV_WEIGHT=0.30
PRED_HEALTH_BEHAVIOUR_WEIGHT=0.30
```

11. Integration Points

11.1 Upstream

Service	Table	Purpose
Feature Engineering (8002)	customer_features	56-column feature vector for ML input
Customer State (8003)	customer_states	State label for health score context

11.2 Downstream

Service	What It Reads	Purpose
API Gateway (8080)	GET /predict/{id}	Customer Detail → Health Score gauge (FR-CUST-02)
API Gateway (8080)	GET /predict/{id}/churn	Churn risk display
Decision Intelligence (8005)	customer_states.health_score	Health-score-gated NBA rules
Frontend	health_score → green/amber/red gauge	Customer Detail screen

11.3 Gateway Route Registration

```
# gateway/app/routes/prediction.py
# Proxy: /api/predict/* → http://prediction-service:8004/predict/*
```

Uses the same httpx proxy pattern as gateway/app/routes/customer_state.py with 502/504 error handling.

12. Implementation Roadmap

Phase 1 — Training + Models (Day 1)

Task	File	Refs
Implement <code>PredictionConfig</code>	<code>app/config/settings.py</code>	§2.1, mirrors <code>FeatureConfig</code>
Implement <code>PredictionRepository</code>	<code>app/repository/repository.py</code>	§9, mirrors <code>FeatureRepository</code>
Implement schemas	<code>app/schemas/schemas.py</code>	§8.2
Implement <code>ChurnPredictor</code>	<code>app/models/churn_predictor.py</code>	§4.5
Implement <code>CLVPredictor</code> (percentile)	<code>app/models/clv_predictor.py</code>	§5.1
Implement <code>HealthScorer</code>	<code>app/services/health_score.py</code>	§6.3
Train initial XGBoost model	<code>scripts/train_models.py</code>	§4.4
Save to <code>models/champion/churn/</code>	—	§7.1
Update <code>models/registry.json</code>	—	§7.2

Phase 2 — API (Day 1-2)

Task	File	Refs
Implement <code>PredictionService</code>	<code>app/services/service.py</code>	mirrors <code>StateService</code>
Implement 6 endpoints	<code>app/api/routes.py</code>	§8.1, mirrors <code>features/routes.py</code>
Wire <code>main.py</code>	<code>main.py</code>	mirrors <code>feature-engineering/main.py</code>

Phase 3 — Integration (Day 2)

Task	File
Run <code>POST /predict/batch</code> on 3 PoC dates	—
Verify <code>customer_states.health_score</code> populated	<code>psql</code>
Gateway route	<code>gateway/app/routes/prediction.py</code>
Integration test	<code>tests/integration/test_predict_api.py</code>

13. Testing Strategy

13.1 Unit Tests

```
# tests/unit/test_health_scorer.py

def test_healthy_customer_low_churn():
```

```

"""churn_prob=0.1, clv_pct=0.8, engagement=80 → high health score."""
scorer = HealthScorer(PredictionConfig())
result = scorer.compute(0.1, 0.8, 80.0)
assert 70 < result["health_score"] <= 100

def test_critical_customer_high_churn():
    """churn_prob=0.9, clv_pct=0.05, engagement=5 → critical."""
    result = scorer.compute(0.9, 0.05, 5.0)
    assert result["health_score"] < 40

def test_null_engagement_defaults_to_50():
    """NULL engagement → neutral 50."""
    result = scorer.compute(0.3, 0.5, None)
    assert result["component_scores"]["behaviour_sub"] == 50.0

def test_score_clamped_to_zero():
    """Negative inputs → clamped to 0."""
    result = scorer.compute(1.5, -0.5, -10)
    assert result["health_score"] == 0.0

def test_score_clamped_to_100():
    """Super-healthy → clamped to 100."""
    result = scorer.compute(-0.5, 2.0, 200)
    assert result["health_score"] == 100.0

```

13.2 Integration Tests

```

# tests/integration/test_predict_api.py

def test_batch_predict_and_verify_backfill():
    """POST /predict/batch → check customer_states.health_score is not NULL."""
    ...

def test_single_customer_prediction():
    """GET /predict/CUST00001?as_of_date=2026-07-27 → 200 with churn + CLV +
health."""
    ...

def test_idempotency():
    """Running predict/batch twice produces identical health_scores."""
    ...

```

14. Decision Log

#	Decision	Rationale	Ref
D1	XGBoost for churn (not LightGBM for PoC)	Simpler API, better default hyperparameters out of the box.	requirements.txt

#	Decision	Rationale	Ref
		LightGBM exists as a configurable fallback.	
D2	Percentile-rank CLV (not trained regressor)	No historical CLV data available. <code>total_amount_90d</code> percentile is a reasonable banking proxy. Production can swap in a regressor without API changes.	§5.2
D3	Health Score in Layer 2 (not Layer 1)	<code>system-design.md</code> §8 places it here. Formula requires <code>churn_prob</code> from ML model which Layer 1 doesn't have.	system-design.md §8
D4	Rule-based training labels for PoC (superseded by D9)	No historical churn data. Originally proposed state-classifier-derived labels; D9 replaced them with account-closure-only labels after the leakage audit revealed recency features would be present in the training vector. Retained for historical context.	§4.2
D5	psycopg2 (sync) — not asyncpg	Mirrors FeatureRepository + StateRepository. Batch scoring is I/O-bound, not compute-bound — async provides no benefit here.	existing repos
D6	Port 8004 (Layer 2 in 8002→8003→8004 sequence)	Matches docker-compose.yml service graph. Gateway routes to all three.	docker-compose.yml
D7	Config pattern: <code>PredictionConfig</code> with env_prefix <code>PRED_</code>	Mirrors <code>FeatureConfig</code> (FE_) and <code>StateConfig</code> (CS_).	§2.1
D8	API pattern: <code>APIRouter(prefix="/predict")</code> + <code>PredictionService</code>	Mirrors <code>/features</code> and <code>/states</code> routers.	§2.3
D9	Account-closure-only labels (Option B — Option A blocked by 10-day date range)	Insufficient forward horizon (need $N \geq 60$, have $N=10$). Account-closure-only produces <30 positive labels — acknowledged PoC limitation. 9 recency-derived features excluded. Expected AUC 0.60-0.72; AUC > 0.90 = leakage signal. Train/test on same population evaluates temporal stability only. Supersedes D4.	§4.2-4.3
D10	Health score weights: <code>shared/config/settings.py</code> is source of truth;	<code>shared/config/settings.py</code> already defines <code>health_score_churn_weight=0.40</code> etc. PredictionConfig re-exposes them	§2.1, §10

#	Decision	Rationale	Ref
	<code>PredictionConfig</code> mirrors with <code>PRED_*</code> overrides	with <code>PRED_</code> prefix for env-var overrideability. Implementation should default to shared config values and only override if <code>PRED_HEALTH_*</code> is explicitly set.	
D11	Batch chunking at 1,000 customers	Prevents memory spikes (>2GB) when scoring 5,000+ customers. 1 chunk = ~500MB peak including XGBoost model + Python overhead. Configurable via <code>PRED_BATCH_CHUNK_SIZE</code> . Each chunk independently scored + backfilled — partial failure isolates to one chunk.	\$4.6
D12	PoC: no calibration — probabilities are ranking scores	<30 positives makes any calibrator (even 2-parameter Platt) prone to overfit. <code>churn_prob</code> is a raw XGBoost score documented as uncalibrated. Health score and Decision Intelligence should use percentile cutoffs, not treat probabilities literally. Production: fit Platt scaling on held-out calibration set when historical labels provide hundreds of positives; store calibrator in registry; <code>ChurnPredictor</code> applies transparently.	\$4.4, \$6.1